Helpyard dashboard redesign and implementation strategy

Prepared: 17 October 2026

Status: UI redesign implemented; automated checks pass; Windows/browser release qualification remains.

Executive summary

Helpyard is a local-first Windows workstation dashboard. Its current application is a single-page HTML/CSS/JavaScript interface served by a Python standard-library HTTP bridge. The bridge exposes Windows system information and explicitly approved actions; it also brokers optional Ollama requests. It is not currently a Python UI-framework application.

The implemented redesign modernizes the Overview, makes performance charts clearer and responsive, shows processor topology, improves the Local AI server and Skills areas, and adds a suggestions shortcut and section-specific homepage recommendations. It preserves the existing browser UI, bridge routes, launcher, authentication model, offline startup, and approval gates. No Python package, frontend package, CDN, font, or image was downloaded or added.

1. Codebase and behavior audit

Confirmed project structure

- `pc-control-dashboard.html`: the browser UI, styles, canvas visualization, navigation, local/cloud chat, settings, and user interactions. The page is a single HTML document with inline JavaScript and CSS.
- `bridge.py`: default local HTTP server and API, implemented using Python's standard library. It binds to `127.0.0.1` and serves the dashboard and system routes.
- `bridge.ps1`: a legacy/manual PowerShell bridge with overlapping routes.
- `Start-Helpyard.bat`: Windows launcher; requires Python 3.10 or newer and starts the Python bridge.
- `tests/`: Python bridge tests, a JavaScript dashboard structure/security check, and a PowerShell legacy-bridge smoke test.
- `resources/`: documentation/catalog and a previously existing modernization-plan PDF. This implementation adds a separate strategy Markdown/PDF pair rather than replacing the existing PDF.

Confirmed capabilities and limitations

- Twelve navigation sections are present: Overview, Projects, Workbench, Processes, Services, Terminal, Local AI, Cloud AI, MCP, Skills, Security, and Settings.
- Appearance already supports dark/light themes, four accent choices, and a preference to show or hide contextual safety tips. Preferences are stored in browser local storage; bridge tokens and cloud API keys are not part of the persisted state.
- Overview metrics come from Windows CIM/PowerShell. Existing CPU activity is Windows `Win32_Processor.LoadPercentage`; memory usage is derived from total and free physical memory. Disk capacity and process count are also reported.
- CPU and memory history is a short, in-memory window collected at five-second intervals while Overview is open. It is not persistent history or a long-term capacity report.

- Process-list CPU is cumulative process CPU time, not current percentage utilization. The UI now identifies this value and its seconds unit instead of calling it CPU utilization.
- Local AI connects to an optional configured Ollama service. Cloud AI makes direct provider requests from the browser and warns that prompts leave the device.
- The Terminal is intentionally disabled; MCP startup is disabled; Skills remain guidance-only examples; and the Security page's audit content is descriptive rather than a persistent event log.
- Python tests stub system/AI operations; passing them does not prove real Windows operations or provider availability.

Security and usability constraints to preserve

Keep loopback-only binding, the short-lived one-use bootstrap code, in-memory bridge bearer token, explicit mutation approvals, request validation/size limits, session-only cloud API keys, disabled arbitrary shell execution, and disabled MCP startup. Do not represent illustrative guidance or an unavailable integration as a live/working capability.

2. UI framework evaluation and decision

Candidate: NiceGUI

NiceGUI is a contemporary Python-first browser UI framework whose component model includes layouts, data binding, events, notifications, and charting. It is a reasonable candidate **if Helpyard chooses a broader Python/ASGI application migration**.

Official sources checked on 17 October 2026:

- NiceGUI documentation describes the browser-based Python GUI, declarative UI, customization, testing, and localhost deployment model.
- PyPI package metadata and project `pyproject.toml` state Python `>=3.10,<4` and an MIT license. The published metadata lists Python 3.10 through 3.14 and marks the project Production/Stable.
- Latest GitHub release was `v3.18.0`, published 7 October 2026. Its release notes include a security fix for uploads to hidden/disabled upload elements.
- Recent repository commits include activity on 16 October 2026, evidence of active maintenance at review time.
- NiceGUI is built on FastAPI/Starlette/Uvicorn and uses a browser frontend built with Vue/Quasar. It therefore brings a dependency tree and frontend runtime resources, unlike the current zero-install design.

Decision: do not install NiceGUI for this redesign

The existing 12-page UI already runs in the browser and communicates with a custom loopback bridge. Adding NiceGUI just to restyle it would introduce an ASGI runtime, transitive dependencies, a second UI-serving model, and a substantial reimplementation of the current navigation, bootstrap, token lifetime, and API interaction. That is not necessary for the requested visual improvements and would increase installation, update, offline-bundling, and security-review work.

The existing Python 3.10+ standard-library bridge and browser-native HTML/CSS/JavaScript/canvas are the chosen implementation. Reconsider NiceGUI only if a separately approved architectural migration is desired. Before adoption, pin an exact reviewed version, inspect and lock transitive dependencies, confirm Python/Windows compatibility, vendor required frontend resources for offline operation, review licenses and security advisories, and re-test bridge authentication and approvals.

3. Implemented design changes

Overview and visualization

- Replaced the original metric row with a clear responsive system overview containing processor activity, memory use, uptime, and process count.

- Added a high-DPI canvas chart with labeled 0–100% grid, responsive resizing, clear CPU/memory keys, a recent time window, gap handling, and a text summary for assistive technologies.
- Kept the chart honest: 5-second samples, at most 24 points (about two minutes), only while the Overview is open, and not retained after reload.
- Added physical core count, logical processor count, and maximum clock-rate fields to the processor panel. Both the Python bridge and legacy PowerShell bridge request these details from CIM.
- Added disk capacity bars with used capacity and percentage labels.
- Updated process-monitor terminology to state that its CPU value is cumulative CPU time in seconds.
- Added homepage suggestions that navigate to the relevant Processes, Local AI, and Settings sections.

Local AI, Skills, and guidance

- Expanded Local AI into a service-status panel, model selector/inventory, endpoint-configuration note, and chat area.
- Distinguished an unavailable Ollama endpoint from a responsive server with no installed models.
- Replaced generic skill placeholders with clearly labeled, non-executable read-only guidance cards linked to the relevant application sections.
- Added a persistent Suggestions shortcut that opens the active page's contextual tip, including when the tips preference was previously disabled.
- Kept the existing theme, accent, and show/hide-tip preferences and did not change the secret-storage policy.

Design and dependency policy

- Used responsive CSS and the existing native canvas API; did not add remote fonts, images, packages, CDN scripts, or build steps.
- No requirements/lockfile, package manifest, or runtime dependency was added. ReportLab was already available in the authoring environment and was used only to generate the PDF; it is not an application dependency.
- No third-party asset or package was downloaded.

4. Verification and remaining work

Automated checks run

- Python bridge unit tests: **7 passed**. Coverage includes auth/bootstrap, route behavior, approval gates, request validation, concurrency behavior, and the processor metric query.
- Default Python bridge live Windows metrics smoke check: **passed** against this machine's Windows CIM/PowerShell, confirming CPU, RAM, fixed disks, and the new processor topology fields.
- Legacy PowerShell bridge smoke suite: **passed**, including the processor topology response fields and existing origin/authentication/approval checks.
- Dashboard structure/security check: **passed**. It verifies all 12 navigation targets, suggestion entries, existing API routes, UI/processor fields, unique DOM IDs, and secret-storage protections.
- The dashboard was opened in the integrated browser as a static local preview. Its Overview rendered, its accessibility snapshot showed the intended sections and controls, and a screenshot confirmed responsive visual composition at the preview viewport.

Still to qualify on Windows

- Test the default `Start-Helpyard.bat` flow in a fresh browser profile, including one-time bootstrap and the real metrics route. The bridge metrics route was smoke-tested directly, but the launcher/bootstrap flow was not exercised end-to-end here.

- Repeat the legacy PowerShell smoke test on any additionally supported Windows versions. The suite passed on this environment.
- Verify live Ollama available/no-model/unavailable states; Ollama was not available for an end-to-end test here.
- Check narrow viewport, keyboard-only navigation, screen-reader behavior, contrast, and browser zoom. The integrated browser's reported viewport size disagreed with the page's CSS viewport during narrow-screen emulation, so mobile overflow is not claimed as verified. The automated DOM check is not a full accessibility audit.
- Compare live utilization to Windows system tools under load; the CPU value remains the CIM load metric and should not be represented as per-core utilization.
- Confirm that disk values and processor topology on representative Windows hardware are returned as expected. These operations were mocked by the Python test suite and were not exercised against a live workstation here.
- Cloud provider calls and real system-changing actions were not executed. Keep mutation approval and provider-key protections in release regression checks.

5. Suggested next stages

Stage	Work	Outcome / acceptance
A. Runtime qualification	Run bridge and PowerShell suites on supported Windows versions; use a fresh browser profile; verify launcher, bridge bootstrap, real metrics, and Ollama states.	Record supported platform results; no regressions in auth, loopback restriction, or approval-required actions.
B. Accessibility and UX review	Keyboard test every section and dialog; review accessible names, chart text alternatives, focus visibility, theme contrast, zoom, and mobile/narrow layout.	No inaccessible primary action; key charts and status have text equivalents; document any remaining WCAG gaps.
C. Metric accuracy	Compare CPU/memory values with Windows tools; clarify metric timing/semantics; gather feedback on whether persistent history is actually needed.	UI labels accurately describe data; any added persistence has an explicit opt-in, retention period, and privacy behavior.
D. Broader framework migration (optional)	Only if independently approved, prototype NiceGUI with existing bridge security constraints, lockfile, vendored assets, and migration tests.	Demonstrate lower maintenance or clear product benefit before replacing the current zero-dependency UI.

Run and review

Start the application by double-clicking `Start-Helpyard.bat`, or run `py -3 .\bridge.py --open-browser` in PowerShell. Keep the bridge window open. For a UI-only static preview, open `pc-control-dashboard.html` directly; system metrics and local integrations will remain unavailable in that mode.

Run checks from the project root:

```
py -3 -m unittest discover -s tests -p test_bridge.py -v
node .\tests\dashboard-check.js
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\tests\bridge-smoke.ps1
```